

Python Garbage Collection Cycle Demonstration

This example demonstrates objects that remain in memory due to a reference cycle even after their original variables are deleted. The cycle is then cleaned up using `gc.collect()`.

```
import sys
import gc

class Node:
    def __init__(self, name):
        self.name = name
        self.link = None

    def __repr__(self):
        return f"Node({self.name})"

# Enable garbage collection
gc.enable()

# -----
# Step 1: Create two Node objects
# -----
A = Node("A")
B = Node("B")

# -----
# Step 2: Create a reference cycle
# A -> B and B -> A
# -----
A.link = B
B.link = A

# Keep temporary references for investigation
tempA = A
tempB = B

# -----
# Step 3: Check reference counts
# -----
print("Before deletion:")
print(f"A refcount: {sys.getrefcount(A)}")
print(f"B refcount: {sys.getrefcount(B)}")

# -----
# Step 4: Delete the original names
# -----
del A
del B

print("\nDeleted variables A and B.")

# The objects still exist because tempA/tempB
# reference them and they reference each other.
print("Objects still accessible through temp references:")
print(tempA, "->", tempA.link)
```

```
print(tempB, "->", tempB.link)

# Remove the last external references
del tempA
del tempB

print("\nRemoved all external references.")

# -----
# Step 5: Investigation
# Objects are now unreachable
# but still exist due to the cycle.
# -----
print("Unreachable objects before collection:")
print(len(gc.garbage), "objects in gc.garbage")

# -----
# Step 6: Force Garbage Collection
# -----
collected = gc.collect()

print("\nGarbage Collector executed.")
print(f"Unreachable objects collected: {collected}")
```